

Sorting code

1. Insertion Sort (C++ Code)

Insertion Sort builds the final sorted array one item at a time by inserting each element into its correct position among the already sorted elements.

```
#include <iostream>
#include <vector>
#include <algorithm>

void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    // Start from the second element (index 1), as the first element is
    for (int i = 1; i < n; ++i) {
        int key = arr[i]; // The element to be inserted
        int j = i - 1;

        // Move elements of arr[0..i-1], that are greater than key,
        // to one position ahead of their current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key; // Place the key in its correct position
    }
}

// Example Usage:
/*
int main() {
    std::vector<int> data = {12, 11, 13, 5, 6};
    insertionSort(data);
    for (int x : data) {
        std::cout << x << " ";
    }
    // Output: 5 6 11 12 13
    return 0;
}
*/
```

2. Bubble Sort (C++ Code)

Bubble Sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```

void bubbleSort(std::vector<int>& arr) {
    int n = arr.size();
    bool swapped;

    // Outer loop controls the number of passes
    for (int i = 0; i < n - 1; ++i) {
        swapped = false;
        // Inner loop compares adjacent elements
        // The last i elements are already in place, so we check up to
        for (int j = 0; j < n - 1 - i; ++j) {
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        // OPTIMIZATION: If no two elements were swapped by inner loop,
        if (swapped == false)
            break;
    }
}

// Example Usage:
/*
int main() {
    std::vector<int> data = {5, 1, 4, 2, 8};
    bubbleSort(data);
    for (int x : data) {
        std::cout << x << " ";
    }
    // Output: 1 2 4 5 8
    return 0;
}
*/

```

3. Selection Sort (C++ Code) 🔍

Selection Sort repeatedly finds the minimum element from the unsorted part and swaps it with the element at the beginning of the unsorted part.

```

#include <iostream>
#include <vector>
#include <algorithm>

void selectionSort(std::vector<int>& arr) {
    int n = arr.size();

    // One by one move the boundary of the unsorted subarray
    for (int i = 0; i < n - 1; ++i) {
        // Find the index of the minimum element in the remaining unsorted
        int min_idx = i;
    }
}

```

```

        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        // Swap the found minimum element with the first element of the
        if (min_idx != i) {
            std::swap(arr[i], arr[min_idx]);
        }
    }
}

// Example Usage:
/*
int main() {
    std::vector<int> data = {64, 25, 12, 22, 11};
    selectionSort(data);
    for (int x : data) {
        std::cout << x << " ";
    }
    // Output: 11 12 22 25 64
    return 0;
}
*/

```

4. Merge sort

```

void merge(int a[], int low, int mid, int high) {
    // subarray1 = a[low..mid], subarray2 = a[mid+1..high], both sorted
    int N = high - low + 1;
    int b[N]; // discuss: why do we need a temporary array b?
    int left = low, right = mid + 1, bIdx = 0;
    while (left <= mid && right <= high) // the merging
        b[bIdx++] = (a[left] <= a[right]) ? a[left++] : a[right++];
    while (left <= mid) b[bIdx++] = a[left++]; // leftover, if any
    while (right <= high) b[bIdx++] = a[right++]; // leftover, if any
    for (int k = 0; k < N; ++k) a[low + k] = b[k]; // copy back
}

void mergeSort(int a[], int low, int high) {
    // the array to be sorted is a[low..high]
    if (low < high) { // base case: low >= high (0 or 1 item)
        int mid = (low + high) / 2;
        mergeSort(a, low, mid); // divide into two halves
        mergeSort(a, mid + 1, high); // then recursively sort them
        merge(a, low, mid, high); // conquer: the merge routine
    }
}

```

5. Counting sort

- **Mechanism:** A non-comparison, **stable** sort effective only for **integers** within a small range (k). It determines the output position of each element by **counting its occurrences** in the input.
- **Process:**
 1. Tally the **frequency** of each unique key in a temporary **Count Array**.
 2. Convert the Count Array into **cumulative counts** (prefix sums), which directly map to the final **starting index** of each key in the sorted output.
 3. Iterate backward through the input to place elements into the correct slot in the output array.
- **Complexity:**
 - Time: $O(n + k)$.
 - Space: $O(n + k)$.
- **Limitation:** Performance degrades significantly if the key range (k) is much larger than the number of elements (n).
- **Primary Use:** When k is small